Assignment 5

IPC and Synchronization

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
04.07.2024

Assignment

Shared and Blocking Buffer Ring

Similar to the shared and blocking stack that was implemented in Tutorial 5, implement a shared and blocking buffer ring.

The buffer ring should respect the following conditions:

1. It must be shared (implement with shared memory)
2. It must be blocking (read and write should wait until they can complete)
3. It must be thread-safe

First, start by completing the `BufferRing` structure. What semaphores and/or mutexes do you need to satisfy the previous conditions? (Use a semaphore initialized to 1 to make a mutex.)

```
struct BufferRing {
    int *buffer;    // internal buffer of the buffer ring
    int idx_reader; // current cursor for the reader
    int idx_writer; // current cursor for the writer
    int size;       // size of the buffer (max. number of elements)

    // complete the structure...
};
```

Next, implement these functions.

```
// Initialize the buffer ring including
// - The internal buffer
// - The synchronization mechanisms
// - The input size represent the number of elements in the buffer
//
// Return the pointer to the newly created buffer ring
struct BufferRing *init_buffer_ring(int size);

// Free the buffer ring's internal buffer and the struct itself
void free_buffer_ring(struct BufferRing *br);

// Read a single value of the buffer ring
// Note that if no value is available, this function should wait until a new
// value has been written
int buffer_ring_read(struct BufferRing *br);

// Write a single value to the buffer ring
// Note that if no more space is available to write, this function should wait
// until a new slot is available in the buffer
void buffer_ring_write(struct BufferRing *br, int value);
```

Shared and Blocking Buffer Ring

Here is a short description for the functioning of a public swimming pool.

- A user gets into the building and changes in a single changing room
- He then ask for a locker to put his clothes,
- He goes take a shower and swim,
- After swimming, he takes another shower before getting his clothes back from the locker
- He dresses again, and then leaves

Follow this protocol and implemment the swimming routine synchronization so that multiple users can go to the swimming pool at the same time without any deadlocks.

```
extern sem_t *changing_room; // initialized to 5
extern sem_t *locker;        // initialized to 20
extern sem_t *shower_room;   // initialized to 12

void swimming_routine() {
    get_changed();

    shower();

    swim();

    shower();

    get_changed();

    leave();
}
```